PROJETO ACADÊMICO — APLICAÇÕES PARA INTERNET

ProductManager — Sistema de Gestão de Produtos

Documentação Técnica Completa da Arquitetura MVC, DAO e API RESTful em Java Puro

Tema do Sistema: Gestão de Estoque e Catálogo de Produtos | **Stack:** Java Puro (HttpServer + JDBC), MySQL, HTML5/CSS3/JS Puro
Data de Conclusão: Novembro/2026 | **Arquitetura:** Padrão MVC com Separação em Controller, Service e DAO

---

1. Resumo Executivo e Propósito do Sistema

O **ProductManager** é uma solução corporativa completa projetada para gerenciar o catálogo mercadológico e o saldo de inventário da empresa **DMAXTECH Systems**. O projeto foi integralmente construído segundo as diretrizes acadêmicas de **Aplicações para Internet**, atendendo aos seguintes pilares de engenharia:

- **Backend em Java Puro (Vanilla Java):** Sem o uso de frameworks como Spring Boot, Hibernate, Spring Data JPA ou Jakarta EE. Todo o controle de ciclo de vida HTTP é realizado através do servidor nativo com.sun.net.httpserver.HttpServer do Java SE.
- **Persistência com JDBC Puro:** Todas as operações de leitura, inclusão, atualização e exclusão no banco de dados MySQL são executadas diretamente via java.sql.Connection, PreparedStatement e ResultSet, garantindo eficiência, isolamento de escopo e proteção contra SQL Injection.
- **Arquitetura MVC com Camadas Obrigatórias:** Divisão estrita em **Controller** (requisições e status HTTP), **Service** (regras de negócio e validações) e **DAO** (acesso atômico ao MySQL).
- **Frontend 100% Puro (Sem Frameworks):** Desenvolvido exclusivamente em HTML5 semântico, CSS3 moderno responsivo e JavaScript Vanilla com a Fetch API nativa. Não requer React, Tailwind CLI, Angular ou Node.js.
- **Segurança e Autenticação Criptográfica:** Login com senhas criptografadas em hash **BCrypt** e emissão de tokens digitais assinados com **HMAC-SHA256** nativo.

2. O que é a API e qual é o seu Papel no Sistema?

A **API RESTful (Application Programming Interface)** é o coração funcional da aplicação. Ela atua como uma interface padronizada que expõe as funcionalidades de negócio através de requisições HTTP seguras, desacoplando completamente a camada de apresentação (View / Navegador) do banco de dados relacional.

Por que a API foi construída e o que ela faz?
1. **Segurança e Integridade:** O navegador nunca se conecta diretamente ao MySQL. A API atua como um guardião central, validando tokens de autorização, verificando credenciais criptografadas e impedindo valores inconsistentes.
2. **Processamento Centralizado de Regras:** A API garante que o catálogo não receba itens com estoque negativo, preços inválidos ou categorias em branco.
3. **Comunicação via Protocolo REST:** Todas as ações correspondem a verbos semânticos do protocolo HTTP (GET para leitura, POST para criação, PUT para edição e DELETE para exclusão).

Como a API funciona nos Containers Docker e Localmente:

No arquivo docker-compose.yml, o sistema está configurado para operar em 3 containers desacoplados que conversam através de uma rede interna virtual (product-manager-net):
• **Container mysql (porta 3306):** Instância do MySQL 8.0 com inicialização automática do banco product_manager via database/init.sql e volume persistente.
• **Container backend (porta 8080):** Executa a nossa API em Java Puro compilada em um Fat JAR executável independente.
• **Container frontend (porta 3000):** Servidor web leve Nginx que entrega os arquivos estáticos (HTML/CSS/JS puro).
Em desenvolvimento local sem Docker, o próprio servidor Java Puro (HttpServer) contém o controlador StaticFileController, servindo o frontend puro diretamente na porta 8080.

3. Modelagem de Dados e Entidades (MySQL)

O modelo relacional é composto por duas entidades essenciais mapeadas de forma limpa em POJOs puros:

Campo	Tipo MySQL	Tipo Java	Descrição e Regras de Validação
id	BIGINT PK	Long	Chave primária numérica gerada automaticamente (AUTO_INCREMENT)
name	VARCHAR(120)	String	Nome do produto. Obrigatório, não nulo, máx. 120 caracteres
description	VARCHAR(500)	String	Descrição e especificações do produto (opcional)
category	VARCHAR(60)	String	Categoria mercadológica. Obrigatória, não nula, máx. 60 caracteres
price	DECIMAL(10,2)	BigDecimal	Preço unitário de venda em R$. Obrigatório e não negativo
stock	INT	Integer	Saldo em estoque. Obrigatório, número inteiro maior ou igual a zero
created_at	DATETIME	LocalDateTime	Timestamp de inserção do registro no banco
updated_at	DATETIME	LocalDateTime	Timestamp da última modificação do registro

Entidade de Segurança (User): Tabela users composta por id (BIGINT PK), username (VARCHAR(60) UNIQUE), password (VARCHAR(255) contendo o hash BCrypt) e role (VARCHAR(20)). A aplicação inclui o seeder automático DataSeeder que garante a existência do usuário administrador padrão (**admin / admin123**) na inicialização.

4. Arquitetura MVC com Separação Obrigatória em Camadas

A aplicação adota com rigor o princípio de responsabilidade única (Single Responsibility Principle) e a divisão em três camadas lógicas independentes:

```
+-------------------------------------------------------------------------+ | CLIENTE HTTP / NAVEGADOR (FRONTEND PURO) | +-----------------------------------+-----------------------------------+ | Requisicoes HTTP (Fetch API) v +-------------------------------------------------------------------------+ | CAMADA CONTROLLER (Web / REST) | | * ProductController.java * AuthController.java * StaticFileController| | - Implementa HttpHandler nativo do Java SE (sem Spring MVC) | | - Mapeia rotas e verbos HTTP (GET, POST, PUT, DELETE, OPTIONS) | | - Extrai e valida o token Bearer no cabecalho Authorization | | - Serializa/desserializa JSON via Gson e emite status semanticos | +-----------------------------------+-----------------------------------+ | Objetos de Dominio / DTOs v +-------------------------------------------------------------------------+ | CAMADA SERVICE (Regras de Negocio) | | * ProductService.java * AuthService.java | | - Validacoes rigorosas de integridade (preco >= 0, estoque >= 0) | | - Autenticacao com BCrypt.checkpw e assinatura HMAC-SHA256 | | - Trata excecoes de dominio (ValidationException, ResourceNotFound) | | - E o UNICO ponto autorizado a acessar o DAO | +-----------------------------------+-----------------------------------+ | Operacoes de Persistencia v +-------------------------------------------------------------------------+ | CAMADA DAO (Data Access Object) | | * ProductDAO.java (Interface) * ProductDAOImpl.java (JDBC) | | * UserDAO.java (Interface) * UserDAOImpl.java (JDBC) | | - Isolamento completo do acesso ao banco de dados MySQL | | - PreparedStatement parametrizados contra SQL Injection | | - Mapeamento explícito de ResultSet para Product e User | +-----------------------------------+-----------------------------------+ | Comandos SQL / Driver JDBC v +-------------------------------------------------------------------------+ | BANCO DE DADOS (MySQL) | | Tabelas: "products" e "users" | +-------------------------------------------------------------------------+
```

Responsabilidades Detalhadas de Cada Componente:

- **ProductDAO / ProductDAOImpl:** Interface e implementação JDBC pura. Todos os métodos (findAll, findById, searchByName, save, update, deleteById) utilizam blocos try-with-resources para encerramento determinístico das conexões. Nenhum comando SQL vaza para as camadas superiores.
- **ProductService:** Centraliza a validação lógica dos dados. Se o usuário tentar cadastrar um item sem nome ou com preço negativo, o serviço monta um mapa detalhado de erros e dispara ValidationException.
- **AuthService:** Realiza a verificação criptográfica da senha através do BCrypt.checkpw. Se as credenciais estiverem corretas, gera um token estruturado assinado com uma chave secreta via Mac.getInstance('HmacSHA256').
- **ProductController & AuthController:** Handlers que implementam a interface com.sun.net.httpserver.HttpHandler nativa do Java, sem necessidade de Servlets tradicionais ou Spring Controllers.

5. Catálogo de Endpoints da API RESTful

A tabela a seguir apresenta todas as rotas expostas pela API RESTful, o método HTTP, a exigência de token e os códigos de resposta semânticos:

Verbo	Rota	Auth	Descrição do Endpoint	Status Sucesso
POST	/api/auth/login	Pública	Autentica operador e gera token HMAC-SHA256	200 OK
GET	/api/products	Bearer	Lista todos os produtos cadastrados no estoque	200 OK
GET	/api/products?search=t	Bearer	Filtra produtos cujo nome contenha o termo 't'	200 OK
GET	/api/products/{id}	Bearer	Retorna os dados detalhados de um produto por ID	200 OK / 404

POST	/api/products	Bearer	Cadastra um novo produto no banco de dados	201 Created
PUT	/api/products/{id}	Bearer	Atualiza todos os dados de um produto existente	200 OK / 404
DELETE	/api/products/{id}	Bearer	Remove permanentemente o produto informado	204 No Content
GET	/docs.html	Pública	Interface visual de documentação interativa da API	200 OK

6. Exemplos de Payload e Respostas da API

A. Autenticação e Login (POST /api/auth/login):

```
// Requisicao enviada: POST /api/auth/login HTTP/1.1 Content-Type: application/json { "username": "admin", "password": "admin123" } // Resposta retornada (200 OK): { "token": "YWRtaW46MTc5NDM2MDI2ODEzNA.oYD8rS2pZf6x7q...", "username": "admin", "role": "ADMIN" }
```

B. Cadastro de Novo Produto (POST /api/products):

```
// Requisicao enviada: POST /api/products HTTP/1.1 Authorization: Bearer YWRtaW46MTc5NDM2MDI2ODEzNA.oYD8rS2pZf6x7q... Content-Type: application/json; charset=utf-8 { "name": "Impressora Multifuncional Laser", "description": "Impressao duplex automatica e conexao Wi-Fi", "category": "Eletronicos", "price": 1499.00, "stock": 7 } // Resposta retornada (201 Created): { "id": 8, "name": "Impressora Multifuncional Laser", "description": "Impressao duplex automatica e conexao Wi-Fi", "category": "Eletronicos", "price": 1499.00, "stock": 7, "createdAt": "2026-11-09T22:24:27", "updatedAt": "2026-11-09T22:24:27" }
```

C. Resposta de Erro de Validação (400 Bad Request):

```
HTTP/1.1 400 Bad Request Content-Type: application/json; charset=UTF-8 { "timestamp": "2026-11-09T22:24:27.152", "status": 400, "error": "Dados Invalidos", "message": "Erro de validacao nos campos informados.", "errors": { "name": "O nome do produto e obrigatorio.", "price": "O preco nao pode ser negativo.", "stock": "O estoque nao pode ser negativo." } }
```

7. Arquitetura do Frontend Puro (Sem Frameworks)

A interface de usuário foi completamente reescrita em tecnologias web fundamentais (Vanilla), eliminando qualquer dependência de Node.js, npm, Webpack, Tailwind ou React:

- **HTML5 Semântico (index.html):** Estrutura leve, acessível e auto-suficiente com ícones vetoriais SVG inline embutidos diretamente no markup.
- **CSS3 Puro (css/style.css):** Utilização de variáveis CSS (Custom Properties) para temas de cores, tipografia fluida, Grid para cards de estatísticas, Flexbox para layouts e modais modernos com efeito de desfoque backdrop-filter.
- **Comunicação Assíncrona (js/api.js):** Módulo em JavaScript puro que consome a API REST através da Fetch API nativa, gerenciando a inclusão automática do token Bearer e interceptando respostas de erro 401 para redirecionamento ao login.
- **Lógica de DOM e Reatividade (js/app.js):** Atualização reativa da tabela, cálculo dinâmico dos 4 cards de métricas em tempo real, busca com *debounce* de digitação, modais de cadastro e edição, confirmação de exclusão e notificações *toast*.

8. Instruções de Instalação e Execução

O sistema pode ser iniciado de duas maneiras flexíveis:

Modo 1 — Execução Local Rápida (sem Docker):

1. Inicie o MySQL local (ex: via WAMP, XAMPP ou serviço nativo na porta 3306).
2. No terminal, execute a aplicação com o Maven Wrapper integrado:
   cd backend
   .\mvnw.cmd test package -DskipTests
   java -jar target\product-manager-1.0.0.jar
3. Acesse o sistema no navegador em **http://localhost:8080/** com o usuário **admin** e senha **admin123**.

Modo 2 — Execução com Docker Compose:
1. Na raiz do projeto, execute: docker compose up --build
2. O Docker orquestrará o MySQL (3306), a API Java Puro (8080) e o Frontend Nginx (3000).

9. Roteiro Sugerido de Apresentação Acadêmica (5 a 8 min)

1. **Introdução (1 min):** Contextualizar o tema (Gestão de Produtos DMAXTECH) e frisar que todo o projeto foi feito em **Java Puro sem frameworks** e **Frontend Puro sem React/Tailwind**.
2. **Demonstração da Aplicação (2 min):** Acessar http://localhost:8080/, demonstrar login seguro com BCrypt, exibição dos cards de métricas em tempo real e busca instantânea.
3. **Demonstração do CRUD (2 min):** Cadastrar um produto novo, mostrar o recálculo dos indicadores, editar o preço ou estoque e realizar a exclusão confirmada por modal.
4. **Inspeção de Código em Camadas (2 min):** Exibir a separação entre ProductController.java (HttpHandler nativo), ProductService.java (regras de validação) e ProductDAOImpl.java (PreparedStatements JDBC puro).
5. **Conclusão (1 min):** Apresentar a documentação visual em http://localhost:8080/docs.html e comprovar a aprovação de 100% dos testes unitários.